

Cloud Native · Platform Engineering · Agentic

Cloud and Kubernetes App

از کوبرنتیز تا پلتفرم Agentic

سلام، من احمدم



احمد

DevOps و زیرساخت

سال‌هاست که در زمینه‌ی دواپس و زیرساخت مشغول فعالیت هستم. از دوران جهان بدون اینترنت تا امروز که عصر هوش مصنوعی است تجربه دارم.

دایرکتور پلتفرم

دیجی‌کالا

داکرمی

از سال ۱۳۹۷

از دنیای بدون اینترنت

تا عصر هوش مصنوعی

مسیر امروز

۱	مسئله: گلوگاه کجاست؟	۵ دقیقه
۲	پایه‌ها: Cloud Native و کوبرنتیز	۱۲ دقیقه
۳	۱۲ فاکتور: قرارداد اپ و پلتفرم	۱۵ دقیقه
۴	زیرساخت سلف‌سرویس و Platform Engineering	۱۵ دقیقه
۵	سرعت توسعه با AI	۱۰ دقیقه
۶	مسیر Agentic	۱۵ دقیقه
۷	Guardrail، اعتماد و امنیت	۸ دقیقه
۸	نقشه راه و جمع‌بندی	۵ دقیقه

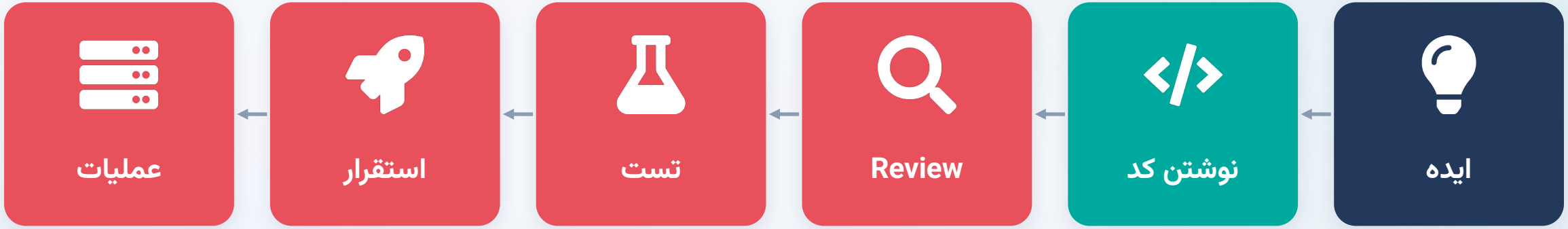


۱

مسئله: گلوگاه کجاست؟

وقتی نوشتن کد ارزان می‌شود، چه چیزی گران می‌ماند

کد سریع‌تر نوشته می‌شود، اما تحویل نه



با AI چند برابر سریع‌تر شده

تقریباً همان سرعت قبلی: صف، انتظار، کار دستی، تأیید

قانون گلوگاه: سرعت کل خط تولید را کندترین مرحله تعیین می‌کند، نه سریع‌ترین.

گلوگاه جابه‌جا شده است

دیروز

گلوگاه اصلی: ظرفیت نوشتن کد

تعداد developerها، سرعت تایپ و دانش فنی تعیین می‌کرد چقدر feature تحویل می‌دهیم.

امروز

گلوگاه اصلی: هر چیزی بعد از کد

بازبینی، محیط تست، استقرار امن، عملیات و مهم‌تر از همه: اعتماد به کدی که خیلی سریع تولید شده.

سؤال امروز: اگر حجم کد ۱۰ برابر شود، پلتفرم ما آماده است؟

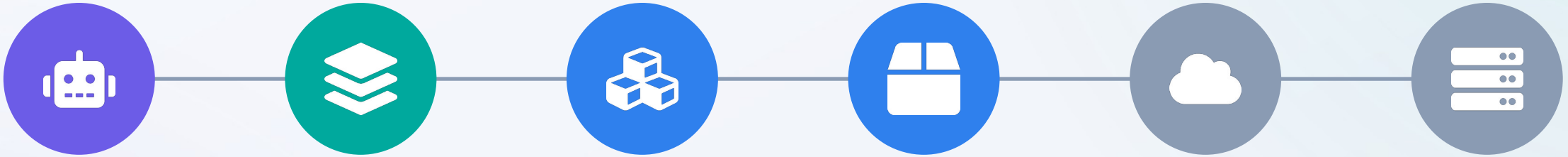


۲

پایه‌ها: Cloud Native و کوبرنتیز

زبان مشترک زیرساخت مدرن

مسیر تحول زیرساخت



سرور فیزیکی

هفته‌ها برای یک سرور

ماشین مجازی

دقیقه‌ها، ولی سنگین

کانتینر

ثانیه‌ها، قابل حمل

کوبرنتیز

ارکستراسیون و خودترمیمی

پلتفرم داخلی

سلف‌سرویس برای انسان

پلتفرم Agentic

سلف‌سرویس برای انسان و ایجنت

هر مرحله یک چیز را ارزان کرد: سخت‌افزار، محیط، استقرار، عملیات... و حالا تصمیم‌گیری و اجرا

کوبرنتیز در یک نگاه

Pod

کوچک‌ترین واحد اجرا: یک یا چند کانتینر با شبکه و فضای مشترک

Deployment

تعداد نسخه‌ها، rolling update و rollback را مدیریت می‌کند

Service

آدرس پایدار و load balancing برای مجموعه‌ای از Podها

Ingress / Gateway

ورود ترافیک بیرونی و مسیریابی HTTP

ConfigMap و Secret

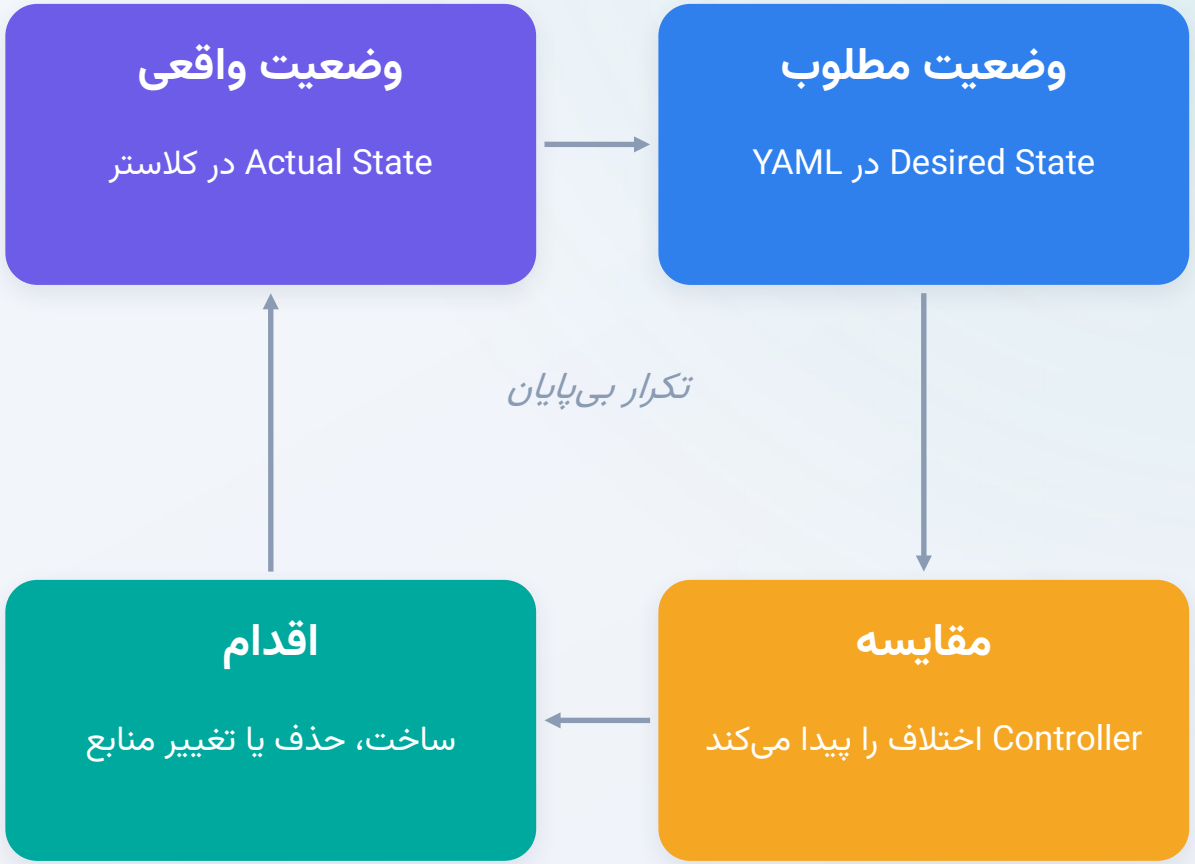
پیکربندی و اطلاعات محرمانه، جدا از image

HPA

افزایش و کاهش خودکار تعداد Podها بر اساس بار

مدل Declarative و حلقه‌ی Reconciliation

- مدل Imperative یعنی «این کارها را به این ترتیب انجام بده»
- مدل Declarative یعنی «من این وضعیت را می‌خواهم»
- سیستم خودش راه رسیدن را پیدا می‌کند و آن را حفظ می‌کند
- نتیجه: خودترمیمی، قابلیت تکرار و قابل بازبینی بودن



چرا این مدل برای ایجنت‌ها ایده‌آل است؟

خروجی ایجنت = متن

ایجنت لازم نیست ده‌ها دستور را درست پشت هم اجرا کند؛ فقط وضعیت مطلوب را می‌نویسد.

قابل بازبینی

هر تغییر یک diff است که انسان یا سیستم دیگری می‌تواند قبل از اعمال آن را بررسی کند.

قابل برگشت

اگر ایجنت اشتباه کند، برگشت به نسخه‌ی قبلی فقط یک revert است.

قابل کنترل

می‌توان روی وضعیت مطلوب policy گذاشت، قبل از آنکه به کلاستر برسد.



۳

۱۲ فاکتور: قرارداد اپ و پلتفرم

اصولی که اپلیکیشن را قابل اجرا، قابل مقیاس و قابل مدیریت می‌کنند

۱۲ فاکتور چیست؟

- یک متدولوژی برای ساخت اپلیکیشن‌های SaaS و سرویس‌های وب
- مستقل از زبان برنامه‌نویسی و فریم‌ورک
- هدف: قابل حمل بودن، مقیاس‌پذیری و استقرار مداوم
- کم کردن فاصله‌ی محیط توسعه و production



۲۰۱۱

منتشر شده توسط تیم Heroku
بر اساس تجربه‌ی اجرای صدها هزار اپلیکیشن

نگاه امروز ما: ۱۲ فاکتور یک «قرارداد» است.
اگر اپ آن را رعایت کند، پلتفرم می‌تواند بدون شناخت داخل اپ آن را اجرا، مقیاس‌دهی و مدیریت کند.

فاکتورهای ۱ تا ۶ روی کوبرنتیز

۱ Codebase

یک codebase در Git، چند استقرار

یک image برای همه‌ی محیط‌ها

۲ Dependencies

وابستگی‌ها صریح و ایزوله

همه چیز داخل container image

۳ Config

پیکربندی در محیط، نه در کد

ConfigMap و Secret

۴ Backing Services

دیتابیس و صف = منبع قابل اتصال

آدرس از طریق Service یا Secret

۵ Build, Release, Run

جدایی کامل سه مرحله

CI ← image + config ← Pod

۶ Processes

پردازه‌های stateless

Deployment؛ state در DB یا PV

فاکتورهای ۷ تا ۱۲ روی کوبرنتیز

۷ Port Binding

سرویس خودش را روی پورت ارائه می‌کند

containerPort + Service

۸ Concurrency

مقیاس افقی با پردازه‌های بیشتر

replicas و HPA

۹ Disposability

شروع سریع، خاموشی امن

SIGTERM، probeها، preStop

۱۰ Dev/Prod Parity

محیط‌ها تا حد ممکن یکسان

یک image + Preview Env

۱۱ Logs

لاگ = جریان رویداد روی stdout

Fluent Bit، Loki، OpenTelemetry

۱۲ Admin Processes

کارهای مدیریتی به‌شکل پردازه‌ی یک‌باره

Job و CronJob

در عمل: Config و Disposability

Config بیرون از image

image یکسان در همه‌ی محیط‌ها؛ فقط ConfigMap و Secret عوض می‌شوند.

readinessProbe

تا اپ آماده نشده، ترافیک دریافت نمی‌کند.

خاموشی امن

preStop و مهلت ۳۰ ثانیه برای تمام کردن درخواست‌های جاری بعد از SIGTERM.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: orders
        image: registry/orders:1.4.2
        envFrom:
        - configMapRef: {name: orders-config}
        - secretRef: {name: orders-db}
        readinessProbe:
          httpGet: {path: /ready, port: 8080}
        lifecycle:
          preStop:
            exec: {command: ["sleep","5"]}
```

فراتر از ۱۲: سه فاکتور مدرن



API First

قرارداد سرویس (OpenAPI، gRPC) قبل از پیاده‌سازی تعریف می‌شود. برای ایجنت‌ها حیاتی است: ایجنت از روی قرارداد کار می‌کند.



Telemetry

متریک، trace و لاگ ساختاریافته از روز اول؛ با استاندارد OpenTelemetry. بدون آن نه انسان و نه ایجنت نمی‌فهمد چه اتفاقی افتاده.



Security

هویت، احراز هویت و مجوز بخشی از طراحی است، نه افزونه. اصل حداقل دسترسی برای سرویس‌ها و ایجنت‌ها.

این سه فاکتور در کتاب Beyond the Twelve-Factor App (کوین هافمن، ۲۰۱۶) در کنار بازنگری ۱۲ فاکتور اصلی مطرح شدند.

۱۲ فاکتور = خوانایی برای ماشین

اگر اپ این را رعایت کند...	...ایجنت می‌تواند این کار را انجام دهد
Config در محیط	ایجنت بدون دست زدن به کد، رفتار را در محیط‌های مختلف تغییر می‌دهد
Stateless و Disposable	ایجنت با خیال راحت restart، scale یا rollback می‌کند
Logs و Telemetry	ایجنت می‌تواند علت مشکل را خودش تحلیل کند
Build/Release/Run جدا	هر تغییر ایجنت یک release مشخص و قابل برگشت است
API First	ایجنت می‌داند سرویس چه کاری می‌کند و چطور صدا زده می‌شود

اپلیکیشنی که برای انسان قابل عملیات است، برای ایجنت هم قابل عملیات است؛ و برعکس.



۴

زیرساخت سلف‌سرویس

Platform Engineering: پلتفرم به‌عنوان محصول

از DevOps کلاسیک به Platform Engineering

DevOps کلاسیک

«You build it, you run it»

هر تیم باید کوبرنتیز، CI/CD، شبکه، امنیت و مانیتورینگ را خودش بلد باشد. نتیجه: بار ذهنی بالا و راه‌حل‌های تکراری و ناهمگون.

Platform Engineering

یک تیم پلتفرم، قابلیت‌های مشترک را به‌شکل سرویس‌های سلف‌سرویس ارائه می‌دهد.

تیم‌های محصول روی منطق کسب‌وکار تمرکز می‌کنند؛ پیچیدگی پشت یک رابط ساده پنهان می‌شود.

هدف: کاهش بار ذهنی (Cognitive Load) تیم‌های محصول، بدون گرفتن استقلال آن‌ها

لایه‌های یک Internal Developer Platform

رابط کاربری و پورتال — پورتال توسعه‌دهنده (مثل Backstage)، CLI، API؛ و حالا: رابط ایجنت (MCP)

Golden Path و قالب‌ها — قالب‌های آماده برای سرویس جدید، pipeline، مستندات

ارکستراسیون و تحویل — GitOps (Argo CD، Flux)، CI، مدیریت منابع ابری (Crossplane، Terraform)

قابلیت‌های مشترک — Observability، امنیت و policy، secret، شبکه، هزینه

زیرساخت — کلاسترهای کوبرنتیز، ابر عمومی یا on-prem، دیتابیس‌ها

Golden Path: راه آسان = راه درست



از «سرویس جدید می‌خواهم» تا «در حال اجرا با مانیتورینگ» در چند دقیقه، نه چند هفته

اختیاری، نه اجباری

تیم می‌تواند از مسیر خارج شود، ولی مسیر آماده آن‌قدر خوب است که دلیلی برای خروج ندارد.

امنیت و استاندارد پیش‌فرض

۱۲ فاکتور، probeها، policyها و telemetry از قبل داخل قالب هستند.

ایجنت‌ها هم از همین مسیر

ایجنتی که سرویس می‌سازد، از همان قالب استفاده می‌کند؛ نه از خلاقیت خودش.

پلتفرم به‌عنوان محصول

کاربر دارد

developerها و حالا ایجنت‌ها مشتری پلتفرم هستند؛ نیازشان را بشناسید.

بازخورد می‌گیرد

نظرسنجی، مصاحبه، و دیدن اینکه کاربران کجا گیر می‌کنند.

نقشه راه دارد

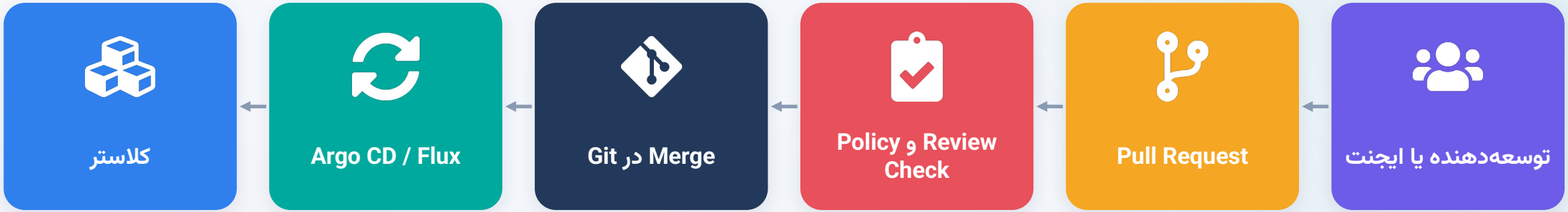
اولویت‌بندی بر اساس ارزش، نه بر اساس ابزارهای جذاب.

اندازه‌گیری می‌شود

میزان استفاده، زمان تا اولین استقرار، رضایت کاربران، DORA.

ضدالگو: پلتفرمی که تیم زیرساخت ساخته ولی هیچ‌کس از آن استفاده نمی‌کند

GitOps: Git به‌عنوان رابط سلف‌سرویس



Pull: کلاستر تغییرات را از Git می‌کشد؛ هیچ‌کس دسترسی مستقیم به کلاستر ندارد

تاریخچه‌ی کامل

هر تغییر: چه کسی، چه زمانی، چرا

تشخیص drift

تغییر دستی در کلاستر خودکار برگردانده می‌شود

rollback ساده

برگشت = revert یک commit

رابط طبیعی ایجنت

ایجنت PR می‌دهد، نه دستور مستقیم

Preview Environment: یک محیط کامل برای هر PR

باز شدن PR

ساخت image

ساخت namespace موقت

آدرس اختصاصی

تست و بازبینی روی محیط واقعی

حذف خودکار بعد از merge

چرا در عصر AI ضروری است؟

- حجم PRها بالا می‌رود؛ یک staging مشترک گلوگاه می‌شود
- بازبین به‌جای خواندن کد، رفتار واقعی را می‌بیند
- ایجنت می‌تواند کار خودش را روی محیط واقعی تست کند
- پیاده‌سازی عملی فاکتور Dev/Prod Parity

ابزارها: Argo CD ApplicationSet، vCluster، Okteto و مشابه

۵

سرعت توسعه با AI

چه چیزی واقعاً سریع‌تر شده و چه چیزی نه

تکامل AI در توسعه‌ی نرم‌افزار

تکمیل خودکار

پیشنهاد خط بعدی کد

دستیار گفتگو

توضیح، تولید تابع، رفع خطا

▼ ما اینجا هستیم

ایجنت کدنویسی

انجام یک تسک کامل روی repo

انسان واگذار می‌کند، AI اجرا می‌کند

سیستم چندایجنتی

ایجنت‌ها در کل چرخه: کد، تست، استقرار، عملیات

انسان هدف و مرز تعیین می‌کند

چه چیزی سریع‌تر شده و چه چیزی نه

سریع‌تر شده

- نوشتن کد boilerplate و CRUD
- نوشتن تست و مستندات
- یادگیری codebase یا تکنولوژی جدید
- refactor و مهاجرت‌های تکراری
- نمونه‌سازی سریع و آزمودن ایده

هنوز کند است

- بازبینی و درک کد تولیدشده
- گرفتن محیط و دسترسی
- تأییدهای دستی و هماهنگی بین تیم‌ها
- استقرار امن و rollback
- تشخیص و رفع مشکل در production

ستون چپ دقیقاً همان چیزهایی است که پلتفرم سلف‌سرویس حل می‌کند

ریسک‌ها: سرعت بدون کنترل

کیفیت

کد به ظاهر درست ولی با خطاهای ظریف؛ تست‌هایی که فقط خودشان را تأیید می‌کنند.

امنیت

وابستگی‌های ناشناخته یا نادرست، secret در کد، تنظیمات ناامن، prompt injection.

مالکیت

کد زیاد با درک کم؛ وقتی مشکل پیش می‌آید، کسی نمی‌داند کد چه می‌کند.

ناهمگونی

هر ایجنت به روش خودش می‌سازد؛ بدون استاندارد، پیچیدگی انفجاری رشد می‌کند.

جواب «کند کردن AI» نیست؛ جواب، guardrail خودکار در پلتفرم است



۶

مسیر Agentic

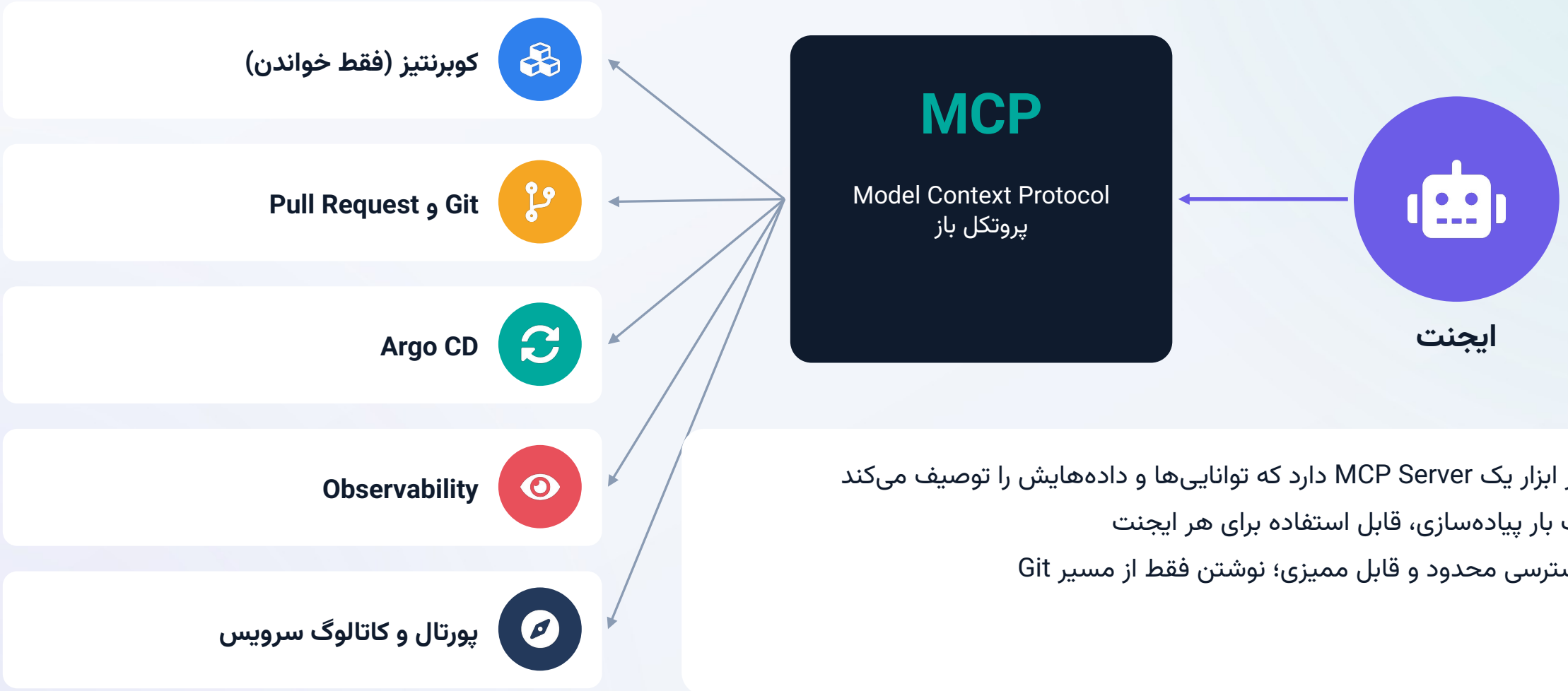
ایجنت‌ها: مصرف‌کننده‌ی جدید پلتفرم

ایجنت: مصرف‌کننده‌ی جدید پلتفرم



ایجنت راه جدیدی نمی‌سازد؛ از همان مسیرهای کنترل‌شده‌ی انسان عبور می‌کند

MCP: رابط استاندارد ایجنت و ابزارها



- هر ابزار یک MCP Server دارد که توانایی‌ها و داده‌هایش را توصیف می‌کند
- یک بار پیاده‌سازی، قابل استفاده برای هر ایجنت
- دسترسی محدود و قابل ممیزی؛ نوشتن فقط از مسیر Git

Agentic SDLC: از تیکت تا production



نقش انسان تغییر می‌کند، حذف نمی‌شود

- تعریف مسئله و معیار موفقیت: تیکت خوب، ورودی ایجنت خوب است
- بازبینی تصمیم‌ها و طراحی، نه هر خط کد؛ با کمک ایجنت بازبین
- مالکیت نهایی: هر سرویس هنوز یک تیم انسانی مسئول دارد

Agentic Ops: ایجنت در عملیات

هشدار

Alert از مانیتورینگ

بررسی

خواندن متریک، لاگ، trace و تغییرات اخیر

فرضیه

تحلیل علت ریشه‌ای با شواهد

پیشنهاد

rollback، scale یا اصلاح کانفیگ به‌شکل PR

تأیید

انسان تصمیم می‌گیرد

اجرا

از مسیر GitOps

کاهش زمان تشخیص

ایجنت در چند دقیقه شواهد را جمع می‌کند و خلاصه‌ی آماده به on-call می‌دهد.

بهینه‌سازی مداوم

تنظیم requests و limits، پیدا کردن منابع بلااستفاده، پیشنهاد صرفه‌جویی هزینه.

Runbook زنده

runbookهای متنی به دستورالعمل‌های قابل اجرا برای ایجنت تبدیل می‌شوند.

چک‌لیست پلتفرم آماده برای ایجنت

رابط ماشینی

API و MCP Server برای قابلیت‌های اصلی پلتفرم

مستندات خوانا برای ماشین

کاتالوگ سرویس، مالک‌ها، قراردادهای API، runbookها

هویت مستقل برای ایجنت

هر ایجنت حساب و مجوز خودش را دارد، نه کلید یک انسان

نوشتن فقط از مسیر Git

تغییر مستقیم در کلاستر ممنوع؛ همه چیز PR است

Telemetry کامل

OpenTelemetry در همه‌ی سرویس‌ها؛ ایجنت بدون داده کور است

پیام‌های خطای واضح

خطای policy باید بگوید چه چیزی اشتباه است و چطور درست شود

سناریوی دمو: ایجنت روی پلتفرم

۱ درخواست

«به سرویس orders یک endpoint برای وضعیت سفارش اضافه کن»

۲ PR از ایجنت

ایجنت کد، تست و تغییر manifest را می‌نویسد و PR باز می‌کند

۳ Policy رد می‌کند

Kyverno: کانتینر بدون resource limits؛ پیام خطای واضح

۴ اصلاح خودکار

ایجنت پیام را می‌خواند، limits را اضافه می‌کند و دوباره push می‌کند

۵ Preview Env

محیط موقت بالا می‌آید؛ ایجنت endpoint را تست می‌کند

۶ تأیید انسان

بازبین نتیجه را روی محیط واقعی می‌بیند و merge می‌کند



۷

Guardrail، اعتماد و امنیت

سرعت و امنیت با هم، نه یکی به‌جای دیگری

لایه‌های دفاعی برای ایجنت‌ها

هویت و RBAC — هر ایجنت هویت مستقل؛ حداقل دسترسی؛ توکن کوتاه‌مدت

Policy as Code — Kyverno یا OPA Gatekeeper: قوانین خودکار روی هر تغییر

امنیت زنجیره‌ی تأمین — SBOM، اسکن آسیب‌پذیری، امضای image با Sigstore/cosign

انسان در حلقه — تأیید انسانی برای تغییرات پرریسک و production

Observability و ممیزی — ثبت کامل: کدام ایجنت، چه تغییری، با چه دلیلی

Policy as Code در عمل

برای همه یکسان

انسان یا ایجنت، قانون یکی است

قانون هم در Git

policyها خودشان review و نسخه‌بندی می‌شوند

پیام خطا = راهنما

ایجنت از پیام می‌فهمد چه چیزی را درست کند

require-limits.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-limits-and-nonroot
spec:
  validationFailureAction: Enforce
  rules:
  - name: check-container
    match:
      any:
      - resources: {kinds: [Pod]}
    validate:
      message: >-
        Set resources.limits and
        securityContext.runAsNonRoot=true
      pattern:
        spec:
          containers:
          - resources:
              limits: {memory: "?*", cpu: "?*"}
            securityContext:
              runAsNonRoot: true
```

سطوح اختیار ایجنت

۰ فقط خواندن

مشاهده، تحلیل، گزارش

مثال: شروع امن؛ مثلاً خلاصه‌ی incident

۱ پیشنهاد

PR باز می‌کند، انسان تصمیم می‌گیرد

مثال: بیشتر کارهای توسعه

۲ اجرا با تأیید

بعد از تأیید انسان، خودکار اجرا می‌کند

مثال: rollback در production

۳ خودمختار در مرز

در محدوده‌ی تعریف‌شده بدون تأیید عمل می‌کند

مثال: محیط dev، scale، به‌روزرسانی وابستگی‌های کم‌ریسک

اعتماد به مرور و بر اساس داده ساخته می‌شود؛ اختیار هر نوع کار جداگانه بالا می‌رود



۸

نقشه راه و جمع‌بندی

از امروز تا پلتفرم Agentic

مدل بلوغ: چهار مرحله

هر مرحله، پیش‌نیاز مرحله‌ی بعد است. پریدن از مرحله‌ی ۱ به ۴ یعنی ایجنت روی زمین سست.

مرحله ۱

Cloud Native

کانتینری‌شدن، کوبرنتیز، رعایت ۱۲ فاکتور، CI پایه

مرحله ۲

سلف‌سرویس

GitOps، Golden Path، پورتال، Preview Environment

مرحله ۳

AI در توسعه

دستیار و ایجنت کدنویسی، بازبینی با کمک AI، policy در CI

مرحله ۴

پلتفرم Agentic

MCP روی پلتفرم، ایجنت در عملیات، سطوح اختیار، ممیزی

اندازه‌گیری: DORA Metrics

تناوب استقرار

Deployment Frequency

هر چند وقت یک بار به production می‌رسیم؟

زمان تحویل تغییر

Lead Time for Changes

از commit تا production چقدر طول می‌کشد؟

نرخ شکست تغییر

Change Failure Rate

چند درصد استقرارها مشکل ایجاد می‌کنند؟

زمان بازیابی

Failed Deployment Recovery Time

بعد از یک استقرار ناموفق، چقدر طول می‌کشد تا برگردیم؟

اگر AI فقط تعداد خطوط کد را بالا ببرد ولی این چهار عدد بهتر نشوند، سریع‌تر نشده‌ایم

از فردا چه کار کنیم؟

ارزیابی ۱۲ فاکتور

سرویس‌های اصلی را با ۱۲ فاکتور بسنجید؛ Disposability و Config معمولاً اولین ضعف‌ها هستند.

GitOps برای یک تیم

یک تیم پایلوت با Argo CD یا Flux؛ دسترسی مستقیم به کلاستر را کم کنید.

سه policy اول

resource limits، non-root، و image از registry مجاز؛ ابتدا در حالت Audit.

اولین ایجنت فقط خواندنی

یک ایجنت که خلاصه‌ی incident را از telemetry آماده می‌کند.

سه پیام اصلی

۱

گلوگاه جابه‌جا شده

AI نوشتن کد را ارزان کرده؛ مزیت رقابتی حالا در پلتفرمی است که این کد را سریع و امن به production می‌رساند.

۲

پایه‌ها هنوز مهم‌اند

۱۲ فاکتور، مدل declarative و GitOps همان چیزهایی هستند که اپ را برای انسان و ایجنت قابل مدیریت می‌کنند.

۳

ایجنت = کاربر جدید پلتفرم

ایجنت‌ها را از همان مسیرهای کنترل‌شده عبور دهید: Golden Path، PR، policy، ممیزی؛ و اختیار را به‌تدریج بدهید.

ما محکوم به رشد هستیم

تکنولوژی منتظر کسی نمی‌ماند؛ تنها راه، بالا نگه داشتن خودمان است

ورزش کنید

بدن سالم، ذهن متمرکز؛ انرژی رشد از اینجا شروع می‌شود

کتاب بخوانید

عمیق فکر کنید؛ فراتر از خلاصه‌ها و ترندهای هفته

یاد بگیرید

هر روز کمی؛ کنجکاوی را خاموش نکنید

یاد بدهید

بهترین راه فهمیدن، توضیح دادن است

رها نکنید؛ وگرنه حذف می‌شوید



پرسش و پاسخ

ممنون از همراهی شما